MPL Work Report

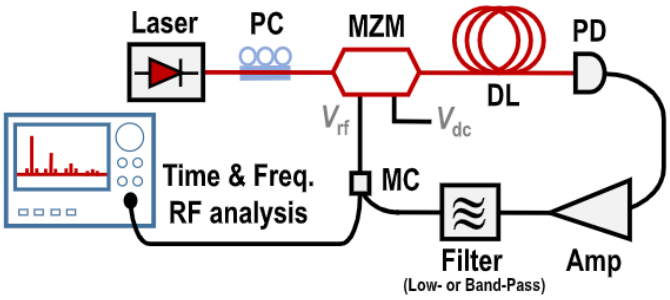
Cyrus Young, May - August, 2025.

Birgit Stiller Lab, Max Planck Institute of Light, Erlangen-Germany

1) Introduction to OEOs:

1.1) Theory:

Imagine we have a Optoelectronic Oscillator (OEO) that is an Ikeda-like system shown below:



Imagine we want to measure the intensity of the optical beam at the point below the MZM (Mach-Zehnder Modulator) and the MC (Microwave coupler). Let's call this point A. Because we are in an oscillator circuit, the intensity of the laser at that point will likely change over time. It would be nice to know how the intensity at that point would change over time. We do this by representing each block/component as a mathematical function.

At point A, we have a signal with intensity $x(t)$. It goes through a MZM which applies a nonlinear function to it, so now after the MZM, we have $f_{NL}(x(t))$. Next, it goes through a delay loop and thus undergoes a time delay so our original signal is now $f_{NL}(x(t-\tau))$. It then goes through a photodetector (which is a measurement device and doesn't affect the signal) and then an amplifier and thus the signal after the amplifier is $\beta f_{NL}(x(t-\tau))$. Lastly, the signal goes through a filter (called h), and now we are back at our starting point, point A (it should be noted that the Microwave Coupler does not have an effect on the device). Therefore, we can represent our system via the following equations:

$$x(t) = h * \beta f_{NL}(x(t-\tau))$$

$$h^{-1} * x(t) = \beta f_{NL}(x(t - \tau))$$

If we set $h^{-1} = \hat{H}$, we get the following equation:

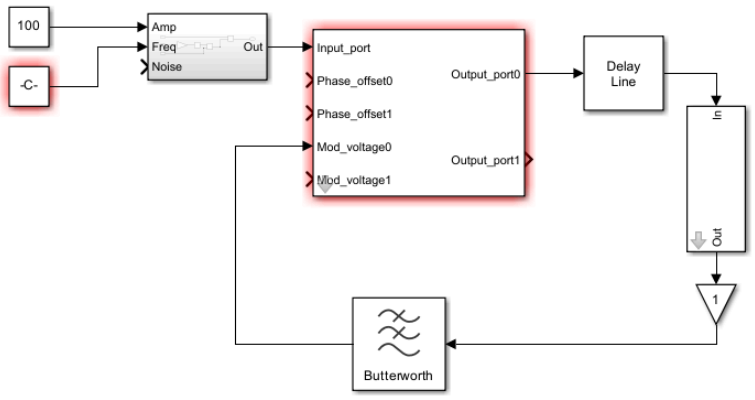
$$\hat{H}[x(t)] = \beta f_{NL}(x(t - \tau))$$

And in many cases, $\hat{H}[x(t)]$ is a differential equation with $x(t)$. This is important as it allows us to give us a mathematical model of how the output (usually the voltage or intensity of the signal) at a given point varies over time. Solving this analytically is in many cases impossible, but it does provide us with the framework to solve it numerically.

1.2) Option 1: Using Simulink to simulate OEO:

The first option is to use Simulink to simulate the behaviour of the OEO as a function of time. This method, as it turns out, is in many ways inferior compared to the next method (labeled Option 2) due to its lack of transparency and lack of versatility. It lacks transparency as the functions are hidden behind blocks - making it harder to detect errors - and lacks versatility as there is not a block for every component and many blocks have a limited set of inputs. In any case, this method is mentioned to show an initial approach.

Below is a Simulink diagram of our original OEO. Although the necessary components are present, making sure that the combination of components acts realistically is a task on its own and would be much easier to verify with our next option.



1.3) Option 2: MATLAB Code to Simulate OEO:

As opposed to using a Simulink simulation to model the system, it would be preferable to represent the system entirely via code. This will allow our representation to not only be more versatile (compared to the simulation which could only simulate a limited number of components) since we can now write code for each component, but also more transparent (as we are now writing the functions directly rather than using blocks). Below is the initial code to simulate an OEO as shown in section 1.1. Although the current code below is shown in the code labeled Iteration 1 in the appendix, this code will undergo changes as time progresses to take into account optimization of the program and varying parameters.

```
clear;  rng default          % deterministic noise seed

%% Core Parameters:

fc   = 10e9;                 % RF freq [Hz]
tauD = 20e-6;                % fibre delay 20 µs
Qrf  = 3000;                 % RF band-pass Q
Glin = 35;                   % small-signal loop gain [dB]
Vpi  = 4;                    % MZM half-wave voltage [V]
phiBias = pi/2;              % quadrature bias for max linearity
Fs = 2e9;   dt = 1/Fs;       % envelope-level sample rate / step
Nt = round(1e-3/dt);         % simulate 1 ms  (>50× τ_D )
Ndelay = round(tauD/dt);     % integer delay in samples
assert(Ndelay>10,'Increase Fs or shorten τ_D');

%% Blocks:

% MZM intensity transfer, bias φb
mzm  = @(v) cos(pi*v/(2*Vpi) + phiBias).^2;

% Soft-limiting RF amplifier (unit saturation, Glin dB small-signal)
A0 = 10^(Glin/20);
amp = @(x) (A0*x)./max(A0,abs(x));     % gentle bend into 1 V max

% Base-band equivalent of RF band-pass (2-pole Butterworth LPF)
BW = fc/Qrf;  f3 = BW/2;  wn = 2*pi*f3;                % rad/s cutoff
[b,a] = butter(2,wn/(pi*Fs));                          % digital LPF

%% Run ------------------------------------------------------------------

bufOpt = zeros(Ndelay,1);              % optical-delay circular buffer
y      = complex(zeros(Nt,1));         % RF output record
noise  = (randn(Nt,1)+1i*randn(Nt,1))*1e-6;

% IIR filter state for per-sample processing
zi = zeros(max(numel(a),numel(b))-1,1);
v_prev = 0;                            % previous RF sample feeding the MZM
```

```
for n = 1:Nt
   % MZM (intensity transfer) ------------------------------------
   v_mzm = mzm(real(v_prev));          % intensity ∝ cos²(⋯)

   % Fibre delay --------------------------------------------------
   k       = mod(n-1,Ndelay) + 1;      % circular-buffer index
   v_del   = bufOpt(k);                % intensity after τ_D
   bufOpt(k) = v_mzm;                  % write current intensity

   % Photo-detector (linear) --------------------------------------
   v_pd = v_del;                       % voltage ∝ delayed intensity

   % RF amplifier -------------------------------------------------
   v_amp = amp(v_pd);                  % soft-limiting gain

   % Narrowband RF filter -----------------------------------------
   [v_filt, zi] = filter(b,a,v_amp,zi);  % 2-pole Butterworth

   % Close the loop -----------------------------------------------
   y(n)   = v_filt + noise(n);         % record & seed with noise
   v_prev = y(n);                      % feeds next-step MZM
end

%% Visualise ----------------------------------------------------------

t = (0:Nt-1)*dt;
Y = fftshift(fft(y));
f = Fs*(-Nt/2:Nt/2-1)/Nt;
figure;
subplot(2,1,1), plot(t*1e3, real(y));
xlabel('Time (ms)'), ylabel('v_{RF} (a.u.)'), title('OEO start-up & steady-state');
subplot(2,1,2), plot(f/1e6, 20*log10(abs(Y)/max(abs(Y))));
xlim([fc/1e6-100 fc/1e6+100]), grid on
xlabel('Frequency (MHz)'), ylabel('Magnitude (dB)'), title('Spectrum near 10 GHz');
```

We will now delve into a detailed explanation of the actual functioning of the code. The first step is to define the core parameters of our system. fc represents our oscillation frequency (i.e. the carrier frequency of which the system is defined). tauD represents the loop delay. Qrf represents the Quality factor. Glin represents the small-signal loop gain - we use a small signal as this describes the gain before saturation. Vpi is the Mach-Zehnder Modulators half-wave voltage - the voltage swing needed to go from maximum to minimum optical output. phiBias represents the DC voltage bias at the MZM. Fs is the envelope-sampling rate. Nt represents the number of time steps needed to represent a certain amount of time in operation (in this case, the number of time steps to simulate 1 ms of operation - which lets you see around 50 round trips). Ndelay represents the number of samples in the delay line. For both Nt and Ndelay, we round the values to ensure that they are implemented in integra number of steps to ensure precision.

```
fc   = 10e9;                % RF freq [Hz]
tauD = 20e-6;               % fibre delay 20 µs
Qrf  = 3000;                % RF band-pass Q
Glin = 35;                  % small-signal loop gain [dB]
Vpi  = 4;                   % MZM half-wave voltage [V]
phiBias = pi/2;             % quadrature bias for max linearity
Fs = 2e9;   dt = 1/Fs;      % envelope-level sample rate / step
Nt = round(1e-3/dt);        % simulate 1 ms  (>50× τ_D )
Ndelay = round(tauD/dt);    % integer delay in samples
```

The next step is to make sure that τ_d, which represents the physical fibre time delay, is sufficiently represented in the program in order for the delay to be non-negligible.

```
assert(Ndelay>10,'Increase Fs or shorten τ_D');
```

Our next operation is performing the corresponding MZM (Mach-Zehnder-Modulator) function on it. Here, the MZM applies a nonlinear effect on the signal, *v*, by applying a cos^2 plus phase operation on it.

```
% MZM intensity transfer, bias φb
mzm  = @(v) cos(pi*v/(2*Vpi) + phiBias).^2;
```

We then need to make sure the signal is amplified, and in a realistic fashion (i.e. we want it to eventually converge to some value, *A0*, as our signal becomes larger (thus mimicking the behavior of actual amplifiers). We do this by giving the following function, *amp(x)*, where *x* is the signal:

```
% Soft-limiting RF amplifier (unit saturation, Glin dB small-signal)
A0 = 10^(Glin/20);
amp = @(x) (A0*x)./max(A0,abs(x));     % gentle bend into 1 V max
```

We then implement a band-pass filter like so. What the following code below implements is a lowpass filter with a frequency band ranging from $0 \leq f \leq f_c / 2Q_{rf}$. The reason it is referred to as “base-band” is that physically, it would be equivalent to a bandpass starting just above 0 Hz, so in MATLAB we implement a low pass filter.

```
% Base-band equivalent of RF band-pass (2-pole Butterworth LPF)
BW = fc/Qrf;  f3 = BW/2;  wn = 2*pi*f3;                % rad/s cutoff
[b,a] = butter(2,wn/(pi*Fs));                          % digital LPF
```

The next step is to create the optical-delay circuit buffer. The first line, bufOpt = zeros(Ndelay,1);, allocates a column vector of length Ndelay to serve as a circular buffer for the optical intensity exiting the MZM. The code overwrites the oldest sample with the newest and reads from that same position rather than shifting an array at every time step - emulating the fixed fiber delay τD. Next, y = complex(zeros(Nt,1)); pre allocates a complex‑valued column vector of length Nt to record the RF loop output at each envelope sample. The reason it is complex from the outset is because this ensures versatility in the program by allowing both real and imaginary components in the output. Finally, noise = (randn(Nt,1) + 1i*randn(Nt,1)) * 1e-6; generates two independent Gaussian random sequences, combines them into a small‑amplitude complex noise vector (on the order of 10^{-6}), and seeds each step of the loop with a perturbation. This tiny noise term breaks the symmetry of the zero‑signal state and kick-starts the oscillation, while preallocating all arrays up front maximizes performance of the program.

```
%% Run ---------------------------------------------------------------

bufOpt = zeros(Ndelay,1);               % optical-delay circular buffer
y      = complex(zeros(Nt,1));          % RF output record
noise  = (randn(Nt,1)+1i*randn(Nt,1))*1e-6;
```

The first line allocates the initial internal state for the digital IIR filter. The second line sets the starting RF envelope voltage feeding into the MZM to 0. This is to ensure that the modulator begins from its unbiased quadrature point and that the filter state and ebay buffier are initially 0.

```
% IIR filter state for per-sample processing
zi = zeros(max(numel(a),numel(b))-1,1);

v_prev = 0;                             % previous RF sample feeding the MZM
```

The loop (which runs for each time step) shows how the optical signal evolves as it goes through each component. It first passes through an MZM - in which a nonlinear function is applied to it. It then passes through the delay loop - the current buffer index k is computed, the intensity after the delay is computed and the original slot for the buffer output is replaced by the new value.

The next step is to represent the photo-detector. In reality, this simply measures the voltage of the signal at that point - thus the code represents this by simply setting the voltage of the photodetector, v_pd, equal to that of the voltage exiting the delay loop. The signal then goes through an amplifier, in which the amp() function defined previously in the program is applied.

The newly amplified signal then goes through the 2-pole Butterworth band-pass (implemented as a low-pass on the baseband envelope) while updating the internal state zi. This selects frequencies near fc and suppresses others. Lastly, some noise is added to our newly filtered signal and the loop is finished. The final value is then used as the initial value for the next loop, indicating the iterative nature of the program.

```
for n = 1:Nt
   % MZM (intensity transfer) ------------------------------------
   v_mzm = mzm(real(v_prev));          % intensity ∝ cos²(⋯)

   % Fibre delay ---------------------------------------------------
   k       = mod(n-1,Ndelay) + 1;      % circular-buffer index
   v_del   = bufOpt(k);                % intensity after τ_D
   bufOpt(k) = v_mzm;                  % write current intensity

   % Photo-detector (linear) ---------------------------------------
   v_pd = v_del;                       % voltage ∝ delayed intensity

   % RF amplifier --------------------------------------------------
   v_amp = amp(v_pd);                  % soft-limiting gain

   % Narrowband RF filter ------------------------------------------
   [v_filt, zi] = filter(b,a,v_amp,zi);  % 2-pole Butterworth

   % Close the loop ------------------------------------------------
   y(n)   = v_filt + noise(n);         % record & seed with noise
   v_prev = y(n);                      % feeds next-step MZM
end
```

Lastly, some final code is needed to visualize the output. The first graph shows the voltage output of the optical signal as a function of time, while the second graph shows the normalized RF spectrum near the carrier frequency of 10 GHz.

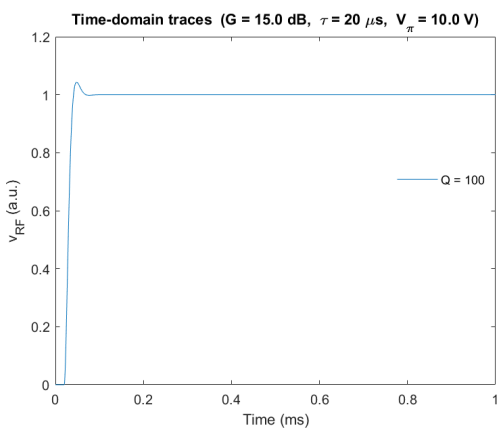
```
%% Visualise ----------------------------------------------------------
t = (0:Nt-1)*dt;
Y = fftshift(fft(y));
f = Fs*(-Nt/2:Nt/2-1)/Nt;

figure;
subplot(2,1,1), plot(t*1e3, real(y));
xlabel('Time (ms)'), ylabel('v_{RF} (a.u.)'), title('OEO start-up & steady-state');
subplot(2,1,2), plot(f/1e6, 20*log10(abs(Y)/max(abs(Y))));
xlim([fc/1e6-100 fc/1e6+100]), grid on
xlabel('Frequency (MHz)'), ylabel('Magnitude (dB)'), title('Spectrum near 10 GHz');
```

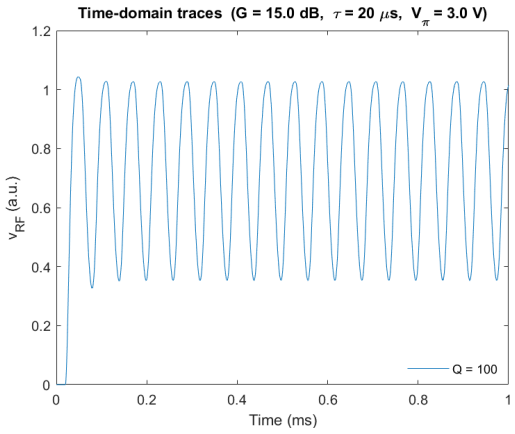
2) Testing of MATLAB program:

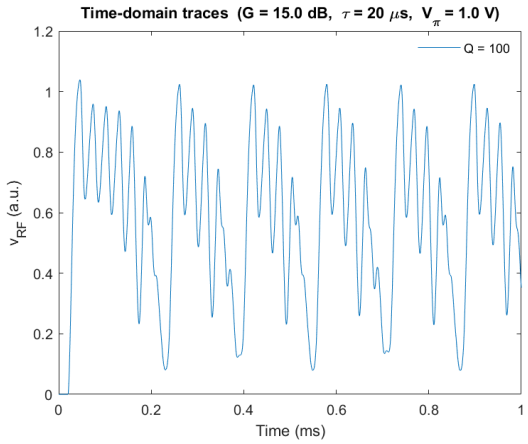
Now that the first iteration of our program has been finished, it is time to start testing it. We start by doing some initial tests, varying the phiBias, Vpi, and gain terms in the MZM, to get a mathematical background of how the system behaves without a filter. Much of the work is detailed in the slides here but the main results are posted below.

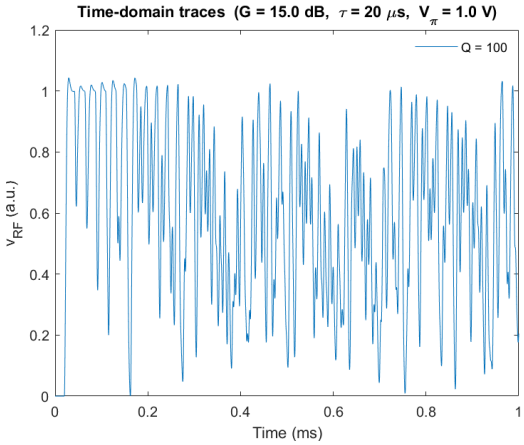
As seen in the appendix, the main program has been updated several times. The mathematics, blocks used, and computational structure of the system itself remains the same. The main changes include changing the bandwidth so that the filter becomes relevant, getting rid of redundant uses of the carrier frequency and altering other parameters (such as the delay, tauD). Such changes can be seen in the codes for Iteration 1 through 3 in the Appendix. Our final iteration for this section and the one which produced the figures below is Iteration 3.

Based on our parameters, there are 4 specific shapes we can get:



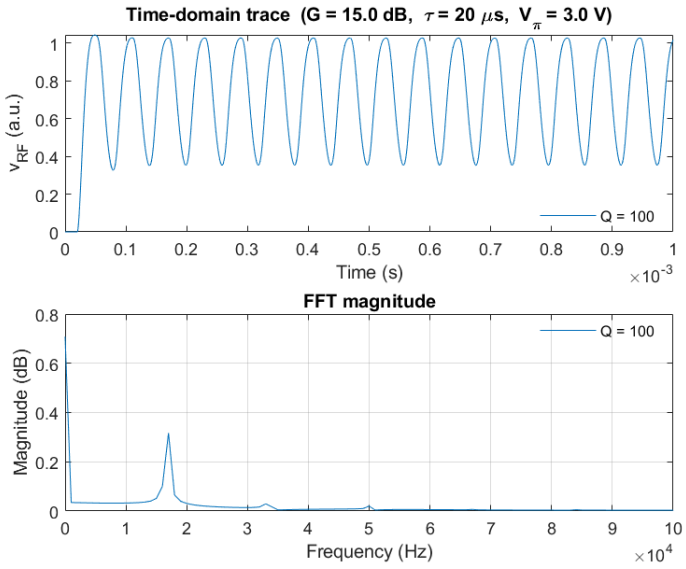


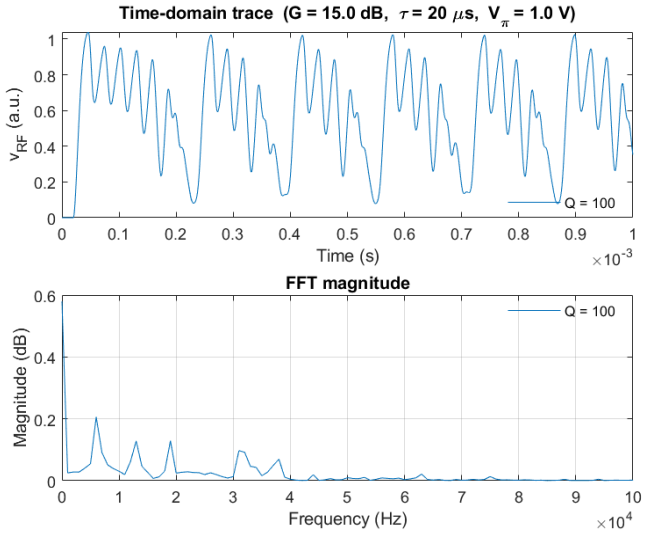


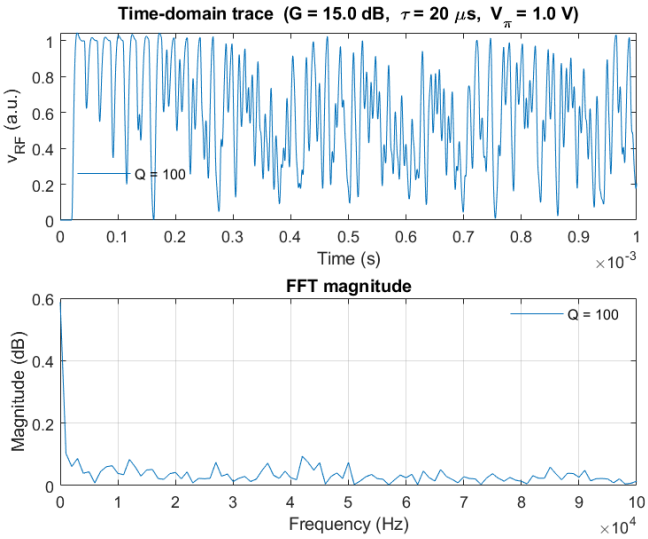


Notice that as we decrease V_π, the half-wave voltage, our signal becomes more chaotic (the difference between the top right and bottom left graph is that half-wave voltage for the bottom left is ⅓ that of the top right). The difference between the bottom left and bottom right graph is not explicitly shown, but resulted from the tripling of the bandwidth (BW) from the bottom left to bottom right graph. This makes sense as a lower value of the BW, for our case, would suppress high-frequency noise - something that is very much present in the bottom right graph.

The frequency spectrum (used by applying the Single Side FFT algorithm) for the 2nd to 4th graphs is shown below:



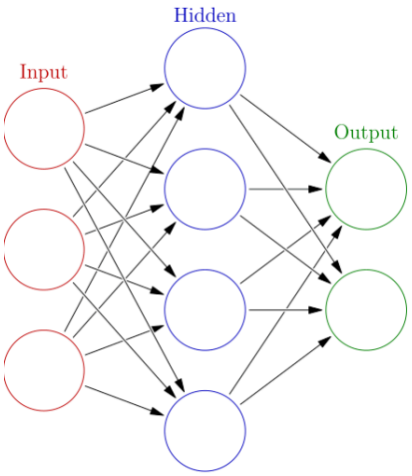



Time-domain trace (G = 15.0 dB, τ = 20 μs, V_π = 1.0 V)
v_RF (a.u.)
Q = 100
Time (s)
×10^-3
FFT magnitude
Magnitude (dB)
Q = 100
Frequency (Hz)
×10^4

3) Creation of Neural Network based on the OEO

3.1) Theory behind the OEO Neural Network:

The next step is to create a Neural Network based on the operation of the OEO. It would be helpful to first explain the structure of our OEO Neural Network itself.



The first layer (or Input layer) is where the inputs of the system would go. Lets represent the input signals into the OEO system by the matrix $\boldsymbol{X}$, where each column of $\boldsymbol{X}$ represents the values corresponding to an individual input signal. An individual input signal, which we can represent as a column vector $\vec{x}_i$ is not strictly defined and could be theoretically anything. For our case, a typical input signal could be a constant 25 mV signal across a timespan of 0.2 ms. Mathematically, we can represent $\boldsymbol{X}$ as:

$$\boldsymbol{X} = [\vec{x}_0\ \vec{x}_1 \dots\ \vec{x}_N]$$

The output of the intermediate layer, or hidden layer, is the output of the input signals once it has traversed the OEO system. Let's mathematically represent the operation of the OEO system as some function $\boldsymbol{f}$. Like the input into the OEO system, the output of the intermediate layer can be represented as a matrix (lets call it $\boldsymbol{H}$) in which each of the columns of said matrix represent a particular output of the system given a particular input. Mathematically:

$$\boldsymbol{H} = [\vec{h}_0\ \vec{h}_1 \dots\ \vec{h}_N] = \boldsymbol{f}(\boldsymbol{X})$$

The final layer (or Output layer) is what gives the output of our Neural Network. In our case, it is a classification scheme that describes the different format of outputs. To receive the values of the final output from the values of the hidden layer, a combination of values and weights are used to help correctly classify the different results of the system. We can use a training set (in which we already know the output beforehand) to tune the particular weights and values that give us the desired classification of our input. The output can also be represented by a matrix which we will call ***O***, where each of the columns of the matrix are a vector that represents a particular classification of the output. We can mathematically represent the weights applied from the hidden layer to the output layer as a matrix W_T . This means that we can represent the operation from which we get the output classification from the hidden layer as:

$$\boldsymbol{O} = [\vec{o}_0\ \vec{o}_1 \ldots\ \vec{o}_N] = W_T\,\boldsymbol{H}$$

Thus, we now have a complete mathematical presentation of our OEO Neural Network. There is one problem however; namely that although we can directly obtain values for ***X*** and ***H***, we cannot directly solve for W_T . As we will see however, this can be resolved by simply deciding our classification scheme *beforehand* and then solving for W_T via the following operation:

$$W_T = \boldsymbol{O} * \boldsymbol{inv(H)}$$

There is no one way to create our matrix for ***O***. The way it is done here is by effectively "correlating" the column vectors of ***O*** with the column vectors of our input ***X***. For example; for our lowest voltage input, the correspond column vector $\vec{o}_0$ might be a column vector with the first entry as 1 and the rest zeros while for our highest voltage input, the correspond column vector $\vec{o}_N$ might be a column vector with the last entry as 1 and the rest zeros.

If this seems a bit abstract, a more concrete example regarding flowers can be given. Imagine we have a dataset of flowers (but we do not know what the flower species are) and for each flower, attributes such as the number of petals, color, diameter, etc are given. The goal of the neural network is to classify the dataset of flowers such that the correct species of flower is predicted for each flower given its attributes. Let's assume that we have two datasets, Set 1 which only contains the attributes of the flowers and Set 2 which contains those attributes AND species of the flower.

We can start by inputting Set 2 into the system for which we can then update the values and weights of the hidden layer such that the number of flowers whose species are correctly guessed

is maximized. We can then input Set 1 into our neural network to then predict the species of flower for each of the flowers.

Back to our OEO system. By training our neural network by inputting different inputs and corresponding outputs, we can then tune our values and weights of the hidden layer to then predict the output of the OEO system only given an input with a high degree of accuracy.

3.2) Explanation of our OEO Neural Network:

The code for our neural network is given below:

```
% ── Parameters ──────────────────────────────────────────────────────────────
Qrf      = 100;          % RF quality factor
Glin_dB  = 15;           % RF amplifier gain (dB)
Vpi      = 1;            % Modulator half-wave voltage (V)

u0_vals  = linspace(0, 9e-2, 10);

t_h  = linspace(20e-6, 220e-6, 100);
H    = zeros(numel(t_h), numel(u0_vals));  % 100 × 10

for k = 1:numel(u0_vals)
   [t_s, y]   = OEO(Qrf, Glin_dB, Vpi, u0_vals(k));   % run oscillator
   H(:,k)     = interp1(t_s, y, t_h, 'linear').';     % 100 × 1
end

O = [1 1 0 0 0 0 0 0 0 0;
    0 0 1 1 1 0 0 0 0 0;
    0 0 0 0 0 1 1 1 0 0;
    0 0 0 0 0 0 0 0 1 1];

M = O*inv(H);
```

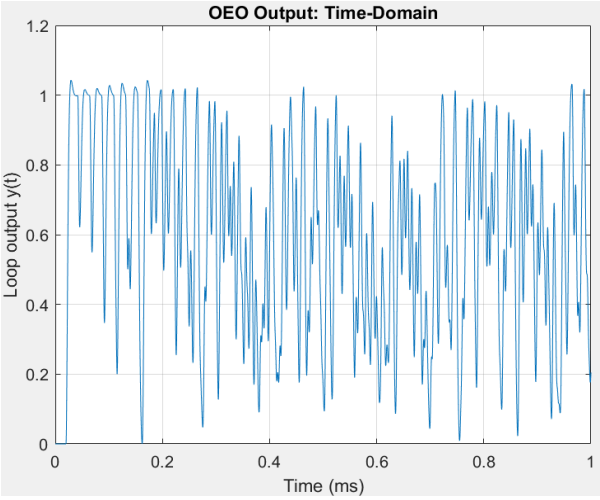
The first three lines describe the input parameters of our system. Qrf affects our bandwidth and thus the operation of the filter of the OEO, Glin_dB affects our gain of the system and Vpi represents the modulator halfwave voltage which can have an effect on both the stability and amplitude of the output of the system (see Section 2 in this document for more details).

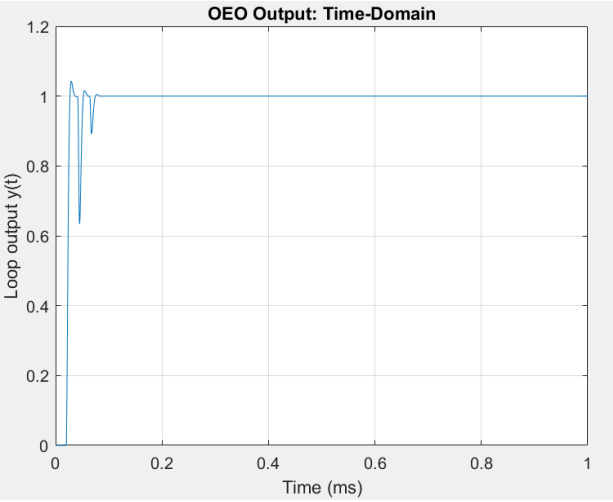
```
Qrf      = 100;          % RF quality factor
Glin_dB  = 15;           % RF amplifier gain (dB)
Vpi      = 1;            % Modulator half-wave voltage (V)
```

The next line is important as it describes the input into our system (effectively the column vectors of ***X***). In this case it is purposefully chosen to represent 10 evenly spaced constant-value signals between 0 mV and 90 mV inclusive.

```
u0_vals  = linspace(0, 9e-2, 10);
```

The reason why it was chosen this way was because the output signal of the OEO starts to immediately behave in a stable manner as soon as the input signal is a constant 90 mV. On the left is the output of our OEO system for an input signal of 0 mV while on the right is the output of our OEO system for an input signal of 90 mV.





The first of the next two lines is important because it allows us to not only control the time we analyze the output but also the number of points in the output. This added versatility to our system allows us to easily change a range of parameters to ensure optimization of the program. The second line essentially creates an empty ***H*** matrix for us in an efficient manner by utilizing previous parameters.

```
t_h  = linspace(20e-6, 220e-6, 100);
H    = zeros(numel(t_h), numel(u0_vals));  % 100 × 10
```

In the block of code below, we are obtaining the values of the matrix ***H*** by calculating each of its column vectors - something that is done by simply taking each of the possible inputs and running them through the OEO system.

```
for k = 1:numel(u0_vals)
   [t_s, y]   = OEO(Qrf, Glin_dB, Vpi, u0_vals(k));   % run oscillator
   H(:,k)     = interp1(t_s, y, t_h, 'linear').';     % 100 × 1
end
```

For our penultimate step, we can build our Output classification matrix ***O*** by creating each of the column vectors within the matrix:

```
O = [1 1 0 0 0 0 0 0 0 0;
    0 0 1 1 1 0 0 0 0 0;
    0 0 0 0 0 1 1 1 0 0;
    0 0 0 0 0 0 0 0 1 1];
```

Lastly, we simply take the matrix product of ***O*** and the *inverse* of ***H*** to get our final matrix W_T (which we call M in the code below).

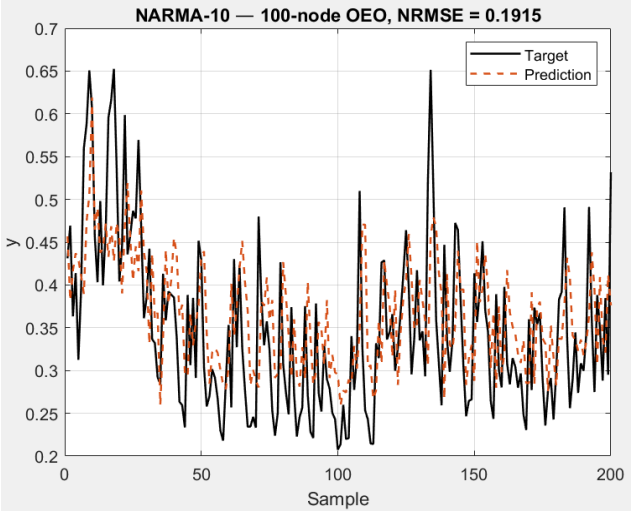
```
M = O*inv(H);
```

4) Testing OEO Neural Network on NARMA10 Series

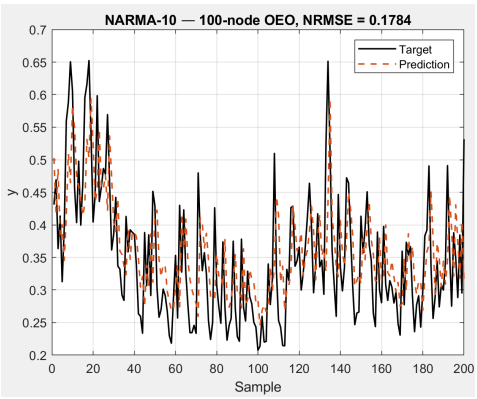
In the program named narma10_oeo_demo.m, we have implemented a delay-based reservoir computing system, with an OEO acting as the single nonlinear node.

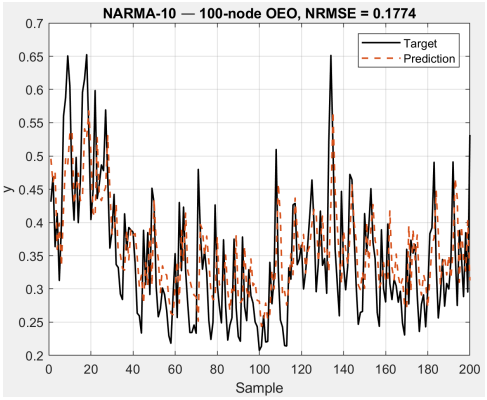
One way we can test the operation of the neural network is to see how well it predicts a NARMA10 (Nonlinear AutoRegressive Moving Average) time series. Via the generation of chaotic and nonlinear patterns, NARMA10 is a time series generator that attempts to mimic the real world behavior of processes. As time series forecasting is crucial in fields such as weather prediction and signal processing, testing the accuracy of our OEO-based neural network on it would be a useful measure to test the networks prediction abilities.

For the NARMA10 prediction algorithm, we are using the code for Iteration 7 for the OEO function and narma10_oeo_demo.m for the main function. In the main function, a mask is also implemented (a normalized mask, the typical mask that we use, corresponds to $\alpha = 1$). With a normalized mask, and a gain (represented by the variable Glin_dB) of 8, we find that our neural network predicts the NARMA10 series like so:

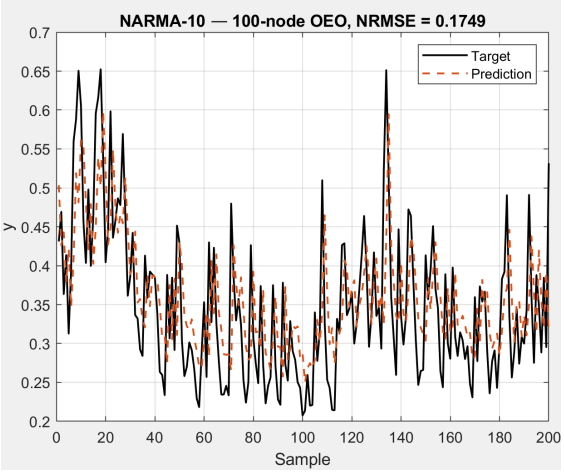


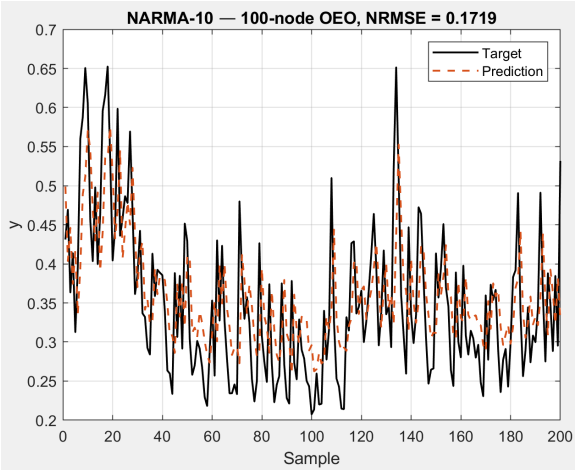
If we increase our gain, the RMSE increases and our prediction worsens. If we continuously decrease our gain to around 3 or 4 while maintaining a normalized mask, we get a slightly better prediction. The left graph is for Glin_dB = 3 while the right is Glin_dB = 4:





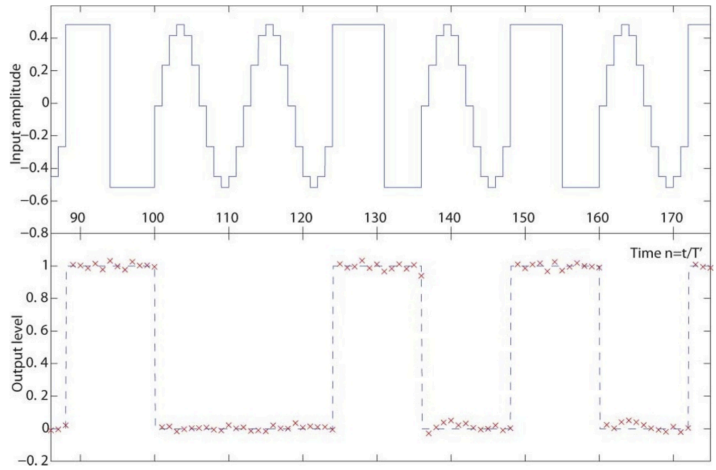
If we decrease to a gain of Glin_dB = 2 (see left graph), we get an even better result. Getting rid of the gain altogether (effectively removing the amplifier in the OEO system), we get our best result yet (see right graph):



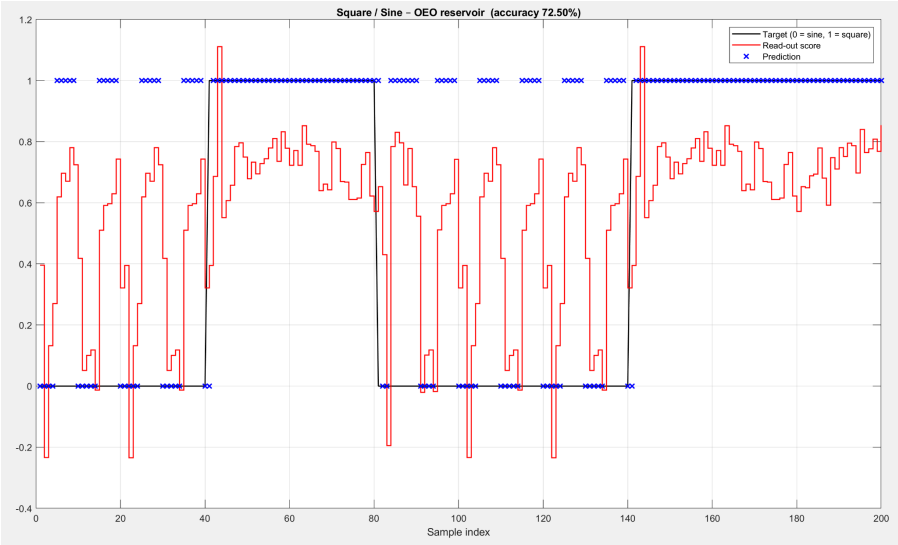


5) Testing OEO Neural Network for Sine-Square Classification:

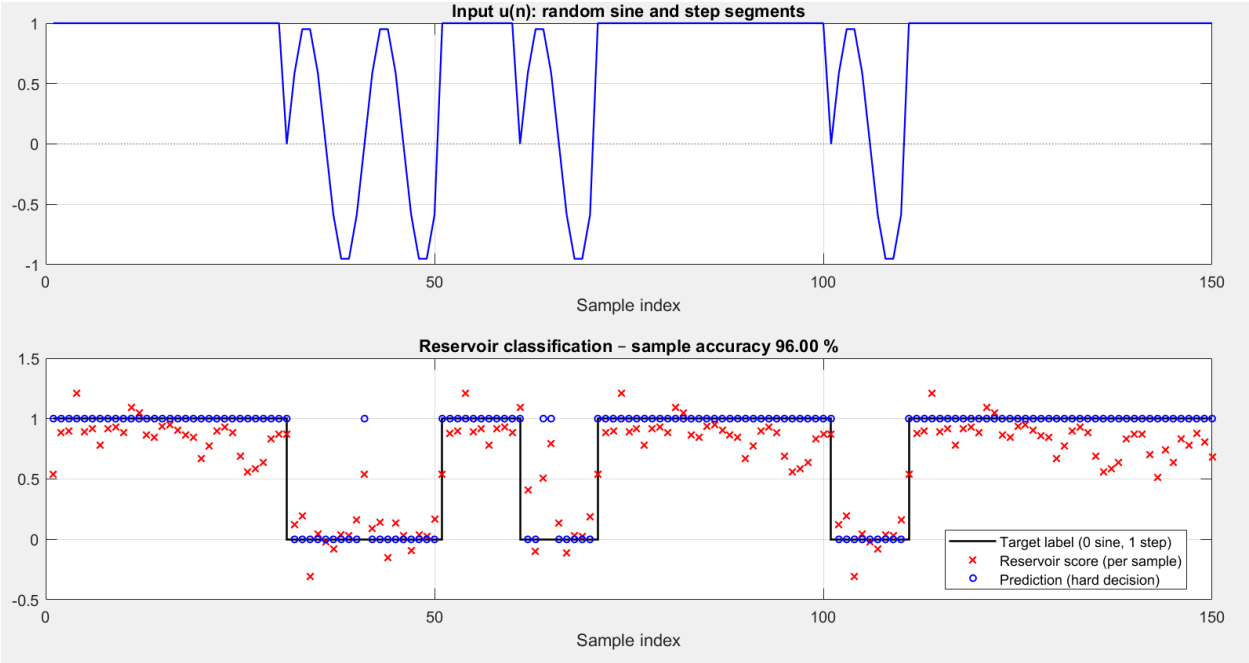
As seen in the paper *Optoelectronic Reservoir Computing*, we want to use our OEO-based Neural Network to predict whether a signal, composed of discretized steps making either a square wave or sine wave, is classified correctly. For example, square wave portions of the input signal are classified as “1” and sine wave portions are classified as “0”.



In the code called [waveClassify_oeo_demo.m], our Neural Network predicts whether a sine or square wave occurs like the example above. However, the code produces a more complicated input signal. (noise is added to it unlike the figure above). The results of the program are shown below:



Once we remove the noise, we can test our new algorithm (see classifySineSquare_NoNoise.m) with a smoother signal to see how the accuracy changes. As we can see below, the accuracy jumps up to 96%.



Relevant Papers:

1) Y. K. Chembo, D. Brunner, M. Jacquot & L. Larger, “Optoelectronic Oscillators with Time-Delayed Feedback,” *Reviews of Modern Physics*, vol. 91, article 035006, 2019. DOI: 10.1103/RevModPhys.91.035006.

2) TD. Brunner, B. Penkovsky, B. A. Marquez, M. Jaquot, I. Fischer & L. Larger, “Tutorial: Photonic Neural Networks in Delay Systems,” *arXiv* 2111.03332 [cs.ET], version 1, 8 Nov 2021.

3) A. Röhm, L. Jaurigue & K. Lüdge, “Reservoir Computing Using Laser Networks,” *IEEE Journal of Selected Topics in Quantum Electronics*, vol. 26, no. 1, article 7700108, Jan./Feb. 2020. DOI: 10.1109/JSTQE.2019.2927578.

4) Y. Paquot, F. Duport, A. Smerieri, J. Dambre, B. Schrauwen, M. Haelterman & S. Massar, “Optoelectronic Reservoir Computing,” *Scientific Reports*, vol. 2, article 287, 27 Feb 2012. DOI: 10.1038/srep00287.

Programs:

Preliminary OEO Programs:

Single Side FFT:

```
function [P1,f] = SingleSideFFT(X,Y)
%UNTITLED6 Summary of this function goes here
%   Detailed explanation goes here
   T = (X(2)-X(1));    % Sampling period
   Fs = 1/T;           % Sampling frequency

   L = length(X);      % Trace length
   L = L - mod(L,2);
   f = Fs/L*(0:(L/2)); % frequency vector

   P0 = fft(Y);
   % P2 = abs(P0/L);
   P2 = (P0/L);
   P1 = P2(1:L/2+1);
   P1(2:end-1) = 2*P1(2:end-1);
end
```

Iteration 1:

```
clear;  rng default          % deterministic noise seed
%% Core Parameters:

fc   = 10e9;                 % RF freq [Hz]
tauD = 20e-6;                % fibre delay 20 µs
Qrf  = 3000;                 % RF band-pass Q
Glin = 1;                   % small-signal loop gain [dB]
Vpi  = 4;                    % MZM half-wave voltage [V]
phiBias = pi/2;              % quadrature bias for max linearity
Fs = 2e9;   dt = 1/Fs;       % envelope-level sample rate / step
Nt = round(1e-3/dt);         % simulate 1 ms  (>50× τ_D )
Ndelay = round(tauD/dt);     % integer delay in samples
assert(Ndelay>10,'Increase Fs or shorten τ_D');

%% Blocks:

% MZM intensity transfer, bias φb
mzm  = @(v) cos(pi*v/(2*Vpi) + phiBias).^2;

% Soft-limiting RF amplifier (unit saturation, Glin dB small-signal)
A0 = 10^(Glin/20);
amp = @(x) (A0*x)./max(A0,abs(x));     % gentle bend into 1 V max

% Base-band equivalent of RF band-pass (2-pole Butterworth LPF)
BW = fc/Qrf;  f3 = BW/2;  wn = 2*pi*f3;                % rad/s cutoff
[b,a] = butter(2,wn/(pi*Fs));                          % digital LPF

%% Run ---------------------------------------------------------------
bufOpt = zeros(Ndelay,1);               % optical-delay circular buffer
y      = complex(zeros(Nt,1));          % RF output record
noise  = (randn(Nt,1)+1i*randn(Nt,1))*1e-6;

% IIR filter state for per-sample processing
zi = zeros(max(numel(a),numel(b))-1,1);
v_prev = 0;                             % previous RF sample feeding the MZM

for n = 1:Nt
   % MZM (intensity transfer) ------------------------------------
   v_mzm = mzm(real(v_prev));          % intensity ∝ cos²(⋯)
   % Fibre delay -------------------------------------------------
   k       = mod(n-1,Ndelay) + 1;      % circular-buffer index
   v_del   = bufOpt(k);                % intensity after τ_D
   bufOpt(k) = v_mzm;                  % write current intensity
   % Photo-detector (linear) -------------------------------------
   v_pd = v_del;                       % voltage ∝ delayed intensity
   % RF amplifier ------------------------------------------------
   v_amp = amp(v_pd);                  % soft-limiting gain
   % Narrowband RF filter ----------------------------------------
   [v_filt, zi] = filter(b,a,v_amp,zi);  % 2-pole Butterworth
   % Close the loop ----------------------------------------------
   y(n)   = v_filt;         % record & seed with noise
```

```
   v_prev = y(n)+noise(n);                          % feeds next-step MZM
end

%% Visualise ------------------------------------------------------------

t = (0:Nt-1)*dt;
Y = fftshift(fft(y));
f = Fs*(-Nt/2:Nt/2-1)/Nt;
figure;
subplot(2,1,1), plot(t*1e3, real(y));
xlabel('Time (ms)'), ylabel('v_{RF} (a.u.)'), title('OEO start-up & steady-state');
subplot(2,1,2), plot(f/1e6, 20*log10(abs(Y)/max(abs(Y))));
xlim([fc/1e6-100 fc/1e6+100]), grid on
xlabel('Frequency (MHz)'), ylabel('Magnitude (dB)'), title('Spectrum near 10 GHz');
```

Iteration 2:

```
clear;  rng default        % deterministic noise seed
%% ------------------------------------------------------------------------

%  Core parameters (delay-line OEO, base-band complex-envelope model)
fc        = 10e6;          % RF centre frequency (Hz)
tauD      = 20e-6;         % fibre delay 20 µs
Qrf       = 3000;          % RF filter Q-factor
Glin_dB   = 2;             % small-signal gain (dB)
Vpi       = 4;             % MZM half-wave voltage (V)
phiBias   = pi/4;          % quadrature bias for linearity
Fs        = 2e9;           % complex-envelope sample-rate (Hz)
dt        = 1/Fs;
simTime   = 1e-3;          % total simulated time (1 ms)
Nt        = round(simTime/dt);
Ndelay    = round(tauD/dt);
assert(Ndelay>10,'Increase Fs or shorten τ_D');

%% ------------------------------------------------------------------------
%  Block definitions
mzm = @(v) cos(pi*v/(2*Vpi) + phiBias).^2;                 % intensity
A0  = 10^(Glin_dB/20);                                     % linear gain
amp = @(x) A0*x ./ max(1,abs(A0*x));                       % 1 V sat
BW  = fc/Qrf;          f3 = BW/2;                          % 3 dB BW
[b,a] = butter(2, 2*f3/Fs);                                % digital LPF
%% ------------------------------------------------------------------------

%  Simulation
bufOpt = zeros(Ndelay,1);          % fibre delay circular buffer
y      = complex(zeros(Nt,1));
noise  = randn(Nt,1)*1e-6;         % real base-band noise
zi = zeros(max(numel(a),numel(b))-1,1);
v_prev = 0;

for n = 1:Nt
   v_mzm        = mzm(real(v_prev));                       % MZM
   k            = mod(n-1,Ndelay)+1;                       % delay index
   v_del        = bufOpt(k);                               % fibre delay
   bufOpt(k)    = v_mzm;
   v_amp        = amp(v_del);                              % RF amp
   [v_filt,zi]  = filter(b,a,v_amp,zi);                    % narrowband
   y(n)         = v_filt;
   v_prev       = y(n) + noise(n);                         % close loop
End

%% ------------------------------------------------------------------------
%  Visualisation
t = (0:Nt-1)*dt;
Y = fftshift(fft(y));
f_bb = Fs*(-Nt/2:Nt/2-1)/Nt;       % base-band axis
f_plot = f_bb + fc;                % centre at 10 MHz
subplot(2,1,1)
```

```
plot(t*1e3, real(y)), xlabel('Time (ms)')
ylabel('v_{RF} (a.u.)'), title('OEO start-up & steady-state')
subplot(2,1,2)
plot(f_plot/1e6, 20*log10(abs(Y)/max(abs(Y))))
xlim([fc/1e6-100 fc/1e6+100]), grid on
xlabel('Frequency (MHz)'), ylabel('Magnitude (dB)')
title('Spectrum near 10 GHz')
```

Iteration 2 test (Multiple Q's):

```
% OEO_Qrf_sweep.m
% Delay-line optoelectronic oscillator – three Q values.
% Legend lists the varying Q; title lists the common parameters.
% ------------------------------------------------------------------------

clear;  rng default
% Sweep list and fixed parameters ---------------------------------------
Qvals   = [50 150 300];     % RF filter Q-factors (varied)
Glin_dB = 2;                % small-signal gain (dB)      (fixed)
Vpi     = 4;                % MZM half-wave voltage (V)   (fixed)
tauD = 20e-6;               % fibre delay (s) (fixed in run_oeo)

% ------------------------------------------------------------------------
figure('Color','w');
ax1 = subplot(1,1,1);  hold(ax1,'on');  box(ax1,'on');

% --- plot traces, attach Q to each legend entry -------------------------
for idx = 1:numel(Qvals)
   Qrf = Qvals(idx);
   [t_ms, y_rf] = run_oeo(Qrf, Glin_dB, Vpi);
   plot(ax1, t_ms, y_rf, ...
        'DisplayName', sprintf('Q = %d', Qrf));
end
% --- legend: now shows only the three Q values --------------------------
legend(ax1,'Interpreter','tex','Location','best','Box','off');
% --- title with common parameters ---------------------------------------
title(ax1, sprintf('Time-domain traces  (G = %.1f dB,  \\tau = %.0f \\mus,
V_{\\pi} = %.1f V)', ...
                   Glin_dB, tauD*1e6, Vpi), ...
     'Interpreter','tex');
xlabel(ax1,'Time (ms)');
ylabel(ax1,'v_{RF} (a.u.)');

% ========================================================================
% Core OEO model (unchanged) ---------------------------------------------
function [t_ms,y_real,f_MHz,Y_dB] = run_oeo(Qrf,Glin_dB,Vpi)
   fc      = 1e7;                  % RF centre frequency (Hz)
   tauD    = 20e-6;                % fibre delay (s)
```

```
    phiBias = pi/4;                 % MZM bias
    Fs      = 2e9;                  % sample rate (Hz)
    dt      = 1/Fs;
    simTime = 4e-4;                 % 0.4 ms total
    Nt      = round(simTime/dt);
    Ndelay  = round(tauD/dt);
    % component models ------------------------------------------------------
    mzm = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
    A0  = 10.^(Glin_dB/20);
    amp = @(x) A0.*x./max(1,abs(A0.*x));
    BW    = fc/Qrf;                 % 3-dB bandwidth
    [b,a] = butter(2,2*(BW/2)/Fs);
    % simulation loop -------------------------------------------------------
    buf   = zeros(Ndelay,1);
    y     = complex(zeros(Nt,1));
    noise = randn(Nt,1)*1e-6;
    zi    = zeros(max(numel(a),numel(b))-1,1);
    v_prev = 0;
    for n = 1:Nt
        v_mzm       = mzm(real(v_prev));
        k           = mod(n-1,Ndelay)+1;
        v_del       = buf(k);
        buf(k)      = v_mzm;
        v_amp       = amp(v_del);
        [v_filt,zi] = filter(b,a,v_amp,zi);
        y(n)        = v_filt;
        v_prev      = y(n)+noise(n);
    end
    % derived outputs -------------------------------------------------------
    t_ms  = (0:Nt-1)*dt*1e3;
    Y     = fftshift(fft(y));
    f_bb  = Fs*(-Nt/2:Nt/2-1)/Nt;
    f_MHz = (f_bb+fc)/1e6;
    Y_dB  = 20*log10(abs(Y)/max(abs(Y)));
    y_real = real(y);
end
```

Iteration 2 test (Single Q-value):

```
% OEO_singleQ.m
% Delay-line optoelectronic oscillator – one Q value, no sweep loop
% ------------------------------------------------------------------------

clear;  rng default
% Fixed parameters ------------------------------------------------------
Qrf     = 100;      % RF filter Q-factor
Glin_dB = 15;       % small-signal gain (dB)
Vpi     = 1;        % MZM half-wave voltage (V)
tauD    = 20e-6;    % fibre delay (s)   (run_oeo constant)

% ------------------------------------------------------------------------
figure('Color','w');
ax1 = subplot(1,1,1);  hold(ax1,'on');  box(ax1,'on');

% --- run model once and plot ------------------------------------------
[t_ms, y_rf] = run_oeo(Qrf, Glin_dB, Vpi);
plot(ax1, t_ms, y_rf, 'DisplayName', sprintf('Q = %d', Qrf));
legend(ax1,'Interpreter','tex','Location','best','Box','off');
title(ax1, sprintf('Time-domain trace  (G = %.1f dB,  \\tau = %.0f \\mus,  V_{\\pi}
= %.1f V)', ...
                   Glin_dB, tauD*1e6, Vpi), ...
     'Interpreter','tex');
xlabel(ax1,'Time (ms)');
ylabel(ax1,'v_{RF} (a.u.)');

% ========================================================================
% Core OEO model (unchanged) ---------------------------------------------
function [t_ms,y_real,f_MHz,Y_dB] = run_oeo(Qrf,Glin_dB,Vpi)
   tauD    = 20e-6;                % fibre delay (s)
   phiBias = pi/4;                 % MZM bias
   Fs      = 2e9;                  % sample rate (Hz)
   dt      = 1/Fs;
   simTime = 1e-3;                 % 1 ms total
   Nt      = round(simTime/dt);
   Ndelay  = round(tauD/dt);
   % component models ----------------------------------------------------
   mzm = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
   A0  = 10.^(Glin_dB/20);
   amp = @(x) A0.*x./max(1,abs(A0.*x));
   BW    = 1/tauD;                 % 3-dB bandwidth
   [b,a] = butter(2,2*(BW/2)/Fs);
   % simulation loop -----------------------------------------------------
   buf   = zeros(Ndelay,1);
   y     = complex(zeros(Nt,1));
   noise = randn(Nt,1)*1e-6;
   zi    = zeros(max(numel(a),numel(b))-1,1);
   v_prev = 0;
   for n = 1:Nt
       v_mzm       = mzm(real(v_prev));
       k           = mod(n-1,Ndelay)+1;
```

```
        v_del       = buf(k);
        buf(k)      = v_mzm;
        v_amp       = amp(v_del);
        [v_filt,zi] = filter(b,a,v_amp,zi);
        y(n)        = v_filt;
        v_prev      = y(n)+noise(n);
    end
    % derived outputs ------------------------------------------------------
    t_ms  = (0:Nt-1)*dt*1e3;
    Y     = fftshift(fft(y));
    f_bb  = Fs*(-Nt/2:Nt/2-1)/Nt;
    Y_dB  = 20*log10(abs(Y)/max(abs(Y)));
    y_real = real(y);
    f_MHz = (f_bb + 1/tauD) / 1e6;   %#-ok<NASGU>
end
```

Iteration 3 test:

```
clear;  rng default

% Fixed parameters ------------------------------------------------------
Qrf      = 100;      % RF filter Q-factor
Glin_dB  = 15;       % small-signal gain (dB)
Vpi      = 1;        % MZM half-wave voltage (V)
tauD     = 20e-6;    % fibre delay (s)
% -----------------------------------------------------------------------

%% ---------- TIME-DOMAIN TRACE ------------------------------------------
figure('Color','w');
ax1 = subplot(2,1,1);  hold(ax1,'on');  box(ax1,'on');
% run model once and plot
[t_s, y] = run_oeo(Qrf, Glin_dB, Vpi);
[Y, f] = SingleSideFFT(t_s, y);
plot(ax1, t_s, y, 'DisplayName', sprintf('Q = %d', Qrf));
legend(ax1,'Interpreter','tex','Location','best','Box','off');
title(ax1, sprintf('Time-domain trace  (G = %.1f dB,  \\tau = %.0f \\mus,  V_{\\pi}
= %.1f V)', ...
                   Glin_dB, tauD*1e6, Vpi), 'Interpreter','tex');
xlabel(ax1,'Time (s)');
ylabel(ax1,'v_{RF} (a.u.)');

%% ---------- FFT MAGNITUDE ----------------------------------------------
ax2 = subplot(2,1,2);  hold(ax2,'on');  box(ax2,'on');
plot(ax2, f, abs(Y), 'DisplayName', sprintf('Q = %d', Qrf));
xlim([0,1e5])
xlabel(ax2,'Frequency (Hz)');
ylabel(ax2,'Magnitude (dB)');
title(ax2,'FFT magnitude');
grid(ax2,'on');
legend(ax2,'Location','best','Box','off');

% ========================================================================
% Core OEO model ---------------------------------------------
function [t_s,y] = run_oeo(Qrf,Glin_dB,Vpi)
   tauD    = 20e-6;                % fibre delay (s)
   phiBias = pi/4;                 % MZM bias
   Fs      = 2e9;                  % sample rate (Hz)
   dt      = 1/Fs;
   simTime = 1e-3;                 % 1 ms total
   Nt      = round(simTime/dt);
   Ndelay  = round(tauD/dt);

   % component models ------------------------------------------------
   mzm = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
   A0  = 10.^(Glin_dB/20);
   amp = @(x) A0.*x./max(1,abs(A0.*x));
```

```
    BW    = 1/tauD;                 % 3-dB bandwidth
    [b,a] = butter(2,2*(BW/2)/Fs);

    % simulation loop ----------------------------------------------------
    buf   = zeros(Ndelay,1);
    y     = complex(zeros(Nt,1));
    noise = randn(Nt,1)*1e-6;
    zi    = zeros(max(numel(a),numel(b))-1,1);
    v_prev = 0;

    for n = 1:Nt
        v_mzm       = mzm(real(v_prev));
        k           = mod(n-1,Ndelay)+1;
        v_del       = buf(k);
        buf(k)      = v_mzm;
        v_amp       = amp(v_del);
        [v_filt,zi] = filter(b,a,v_amp,zi);
        y(n)        = v_filt;
        v_prev      = y(n)+noise(n);
    end
    t_s   = (0:Nt-1)*dt;
end
```

Iteration 4:

```
function [t_s,y] = OEO(Qrf,Glin_dB,Vpi,u0)
  rng default
  if nargin<4, u0 = 0; end                % default: no injection
  tauD    = 20e-6;
  phiBias = pi/4;
  Fs      = 2e9;
  dt      = 1/Fs;
  simTime = 1e-3;
  Nt      = round(simTime/dt);
  Ndelay  = round(tauD/dt);
  mzm = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
  A0  = 10.^(Glin_dB/20);
  amp = @(x) A0.*x./max(1,abs(A0.*x));
  BW    = 3/tauD;
  [b,a] = butter(2,2*(BW/2)/Fs);
  buf   = zeros(Ndelay,1);
  y     = zeros(Nt,1);
  noise = randn(Nt,1)*1e-6;
  zi    = zeros(max(numel(a),numel(b))-1,1);
  v_prev = 0;
  for n = 1:Nt
      v_mzm       = mzm(v_prev);            % optical modulation
      k           = mod(n-1,Ndelay)+1;
      v_del       = buf(k);                 % retrieve delayed sample
      buf(k)      = v_mzm;                  % store new sample
      v_amp       = amp(v_del);             % RF amplifier
      [v_filt,zi] = filter(b,a,v_amp,zi);   % RF filter
      y(n)        = v_filt;                 % loop output
      v_prev      = y(n) + noise(n) + u0;   % ← constant injection
  end
  t_s = (0:Nt-1)*dt;
end
```

Iteration 5 (includes a decay constant for exponential input):

```
function [t_s,y] = OEO(Qrf,Glin_dB,Vpi,u0,decayTau)
% OEO  Opto-electronic oscillator with optional exponentially-decaying drive
 rng default
 if nargin < 4,  u0 = 0;        end            % default: no injection
 if nargin < 5,  decayTau = inf; end           %  ∞   ⇒ constant drive
 tauD    = 20e-6;
 phiBias = pi/4;
 Fs      = 2e9;
 dt      = 1/Fs;
 simTime = 1e-3;
 Nt      = round(simTime/dt);
 Ndelay  = round(tauD/dt);
 mzm = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
 A0  = 10.^(Glin_dB/20);
 amp = @(x) A0.*x./max(1,abs(A0.*x));
 BW    = 3/tauD;
 [b,a] = butter(2,2*(BW/2)/Fs);
 buf   = zeros(Ndelay,1);
 y     = zeros(Nt,1);
 noise = randn(Nt,1)*1e-6;
 zi    = zeros(max(numel(a),numel(b))-1,1);
 v_prev = 0;

 for n = 1:Nt
     v_mzm       = mzm(v_prev);                    % optical modulation
     k           = mod(n-1,Ndelay)+1;
     v_del       = buf(k);                         % delayed sample
     buf(k)      = v_mzm;                          % store new sample
     v_amp       = amp(v_del);                     % RF amplifier
     [v_filt,zi] = filter(b,a,v_amp,zi);           % RF filter
     y(n)        = v_filt;                         % loop output
     % ---------- decaying-exponential drive ----------------------------
     u_k         = u0*exp(-((n-1)*dt)/decayTau);    % u_k = u0·e^{-t/τ}
     % ------------------------------------------------------------------
     v_prev      = y(n) + noise(n) + u_k;
 end

t_s = (0:Nt-1)*dt;

end
```

Iteration 6 (Includes mask function but no decay constant):

```
function [t_s,y] = OEO(Qrf,Glin_dB,Vpi,u0)

 rng default
 if nargin<4, u0 = 0; end                % unchanged (default injection)
 tauD    = 20e-6;
 phiBias = pi/4;
 Fs      = 2e9;
 dt      = 1/Fs;
 simTime = 1e-3;
 Nt      = round(simTime/dt);
 % ——------------------------ vector-support for u0 -------------------- %
 if isscalar(u0)                                          % ← added
     u0 = u0*ones(Nt,1);                                  % ← added
 else                                                     % ← added
     if numel(u0) ~= Nt                                   % ← added
         error('u0 vector must have length Nt = %d.',Nt); % ← added
     end                                                  % ← added
     u0 = u0(:);                                          % ← added (column)
 end                                                      % ← added
 %
_____________________________________________________________________________
_______ %
 Ndelay  = round(tauD/dt);
 mzm = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
 A0  = 10.^(Glin_dB/20);
 amp = @(x) A0.*x./max(1,abs(A0.*x));
 BW    = 3/tauD;
 [b,a] = butter(2,2*(BW/2)/Fs);
 buf   = zeros(Ndelay,1);
 y     = zeros(Nt,1);
 noise = randn(Nt,1)*1e-6;
 zi    = zeros(max(numel(a),numel(b))-1,1);
 v_prev = 0;
 for n = 1:Nt
     v_mzm       = mzm(v_prev);
     k           = mod(n-1,Ndelay)+1;
     v_del       = buf(k);
     buf(k)      = v_mzm;
     v_amp       = amp(v_del);
     [v_filt,zi] = filter(b,a,v_amp,zi);
     y(n)        = v_filt;
     v_prev      = y(n) + noise(n) + u0(n);   % ← changed (vector injection)
 end
 t_s = (0:Nt-1)*dt;
end
```

Iteration 7:

```
function [t_s,y] = OEO(Qrf,Glin_dB,Vpi,u0)
% Optoelectronic oscillator – vector-compatible version
   rng default
   if nargin<4, u0 = 0; end
   tauD    = 20e-6;              % loop delay
   phiBias = pi/4;               % MZM bias
   Fs      = 2e9;                % sample rate
   dt      = 1/Fs;
   % run length adapts to u0
   if isscalar(u0)
       simTime = 1e-3;           % default 1 ms
       Nt = round(simTime/dt);
       u0 = u0*ones(Nt,1);
   else
       Nt = numel(u0);
   end
   Ndelay  = round(tauD/dt);     % samples per loop
   mzm   = @(v) cos(pi*v/(2*Vpi)+phiBias).^2;
   A0    = 10^(Glin_dB/20);
   amp   = @(x) A0.*x./max(1,abs(A0.*x));
   BW      = 3/tauD;             % 1-pole equivalent
   [b,a]   = butter(2,2*(BW/2)/Fs);           % 2-pole Butterworth
   buf     = zeros(Ndelay,1);
   y       = zeros(Nt,1);
   noise   = randn(Nt,1)*1e-6;
   zi      = zeros(max(numel(a),numel(b))-1,1);
   v_prev  = 0;
   for n = 1:Nt
       v_mzm       = mzm(v_prev);
       k           = mod(n-1,Ndelay)+1;
       v_del       = buf(k);
       buf(k)      = v_mzm;
       v_amp       = amp(v_del);
       [v_filt,zi] = filter(b,a,v_amp,zi);
       y(n)        = v_filt;
       v_prev      = y(n) + noise(n) + u0(n);
   end
   t_s = (0:Nt-1)'*dt;
end
```

OEO-based Neural Network Programs:

Neural Network Test:

```
% ── Parameters ──────────────────────────────────────────────────────────

Qrf      = 100;          % RF quality factor
Glin_dB  = 15;           % RF amplifier gain (dB)
Vpi      = 1;            % Modulator half-wave voltage (V)

u0_vals  = linspace(0, 9e-2, 10);

t_h  = linspace(20e-6, 220e-6, 100);
H    = zeros(numel(t_h), numel(u0_vals));  % 100 × 10

for k = 1:numel(u0_vals)
   [t_s, y]   = OEO(Qrf, Glin_dB, Vpi, u0_vals(k));   % run oscillator
   H(:,k)     = interp1(t_s, y, t_h, 'linear').';     % 100 × 1
end

O = [1 1 0 0 0 0 0 0 0 0;
    0 0 1 1 1 0 0 0 0 0;
    0 0 0 0 0 1 1 1 0 0;
    0 0 0 0 0 0 0 0 1 1];

M = O*inv(H);
```

Neural Network Test (with time Multiplexing):

```
%% ── User-adjustable parameters ──────────────────────────────────
Qrf      = 100;        % RF quality factor
Glin_dB  = 15;         % RF amplifier gain (dB)
Vpi      = 1;          % Modulator half-wave voltage (V)
u0_vals  = linspace(0,9e-2,10);   % raw data symbols  u_k  (10 clock cycles)
Nv       = 5;                     % virtual nodes per cycle  (mask length)
mask     = [+1 -1 +1 -1 +1];      % binary ±1 mask of length Nv
theta    = 10e-6;                 % time per virtual node θ
t_pick   = theta;                 % pick the sample exactly at t = θ

% quick sanity check
assert(numel(mask)==Nv,'mask length must equal Nv');
%% ── Pre-allocate H:  Nv × (#cycles) ─────────────────────────────
H = zeros(Nv, numel(u0_vals));
%% ── Main loop: run OEO once per mask segment ────────────────────
for k = 1:numel(u0_vals)          % loop over clock cycles (raw symbols)
   for v = 1:Nv                  % loop over segments inside the mask
       u_inj   = u0_vals(k) * mask(v);        % masked injection J_seg
       [t_s,y] = OEO(Qrf, Glin_dB, Vpi, u_inj); % same 4-arg signature
       H(v,k)  = interp1(t_s, y, t_pick, 'linear'); % sample at t = θ
   end
end
%% ── Your read-out (unchanged) ──────────────────────────────────
O = [1 1 0 0 0 0 0 0 0 0;
    0 0 1 1 1 0 0 0 0 0;
    0 0 0 0 0 1 1 1 0 0;
    0 0 0 0 0 0 0 0 1 1];
M = O / H;  %   (use regularisation if H is ill-conditioned)
```

NARMA10 Test: narma10_oeo_demo.m

```
%% NARMA-10 prediction with a 100-node OEO reservoir – *continuous-run version*
clear; close all;
%% 1  Data ------------------------------------------------------
Ntotal = 1000;                        % samples to predict
[~, y] = makeNARMA10(Ntotal, 42);     % reproducible series
Ntrain = 800;   Ntest = Ntotal - Ntrain;
y_train = y(1:Ntrain);
y_test  = y(Ntrain+1:end);
%% 2  Reservoir parameters ---------------------------------------
Qrf      = 100;
Glin_dB  = 0;          % avoids hard clipping but keeps cos² nonlinear
Vpi      = 1;
tauD     = 20e-6;      Fs = 2e9;
dt       = 1/Fs;
Ndelay   = round(tauD/dt);           % 40 000 samples
Nvirt    = 100;      group_len = Ndelay / Nvirt;   % 400 samples/node
rng(1)
alpha = 1;                           % input swing
mask  = alpha * sign(randn(Nvirt,1));  % ±α mask – key for richness
%% 3  Build one long masked-input waveform ----------------------
% Each scalar sample becomes a 40 000-point block
drive = zeros(Ndelay*Ntotal,1);
for k = 1:Ntotal
  idx = (k-1)*Ndelay + (1:Ndelay);
  drive(idx) = encodeInput(y(k), mask, group_len);
end
%% 4  Run the OEO once over the whole drive ---------------------
[t_s, y_res] = OEO(Qrf, Glin_dB, Vpi, drive);   % single continuous simulation
%% 5  Slice reservoir states at virtual-node taps ---------------
node_idx = (0:Nvirt-1)*group_len + ceil(group_len/2);  % 1-based indices
Ymat = reshape(y_res, Ndelay, Ntotal);    % one column per scalar input
Xall = Ymat(node_idx, :);                 % size: Nvirt × Ntotal
Xtrain = Xall(:, 1:Ntrain);
Xtest  = Xall(:, Ntrain+1:end);
%% 6  Ridge read-out (+ bias) ----------------------------------
ridge = 1e-6;
W = (y_train.' * [Xtrain; ones(1,Ntrain)].') / ...
  ([Xtrain; ones(1,Ntrain)] * [Xtrain; ones(1,Ntrain)].' + ...
   ridge*eye(Nvirt+1));
%% 7  Predict ---------------------------------------------------
y_hat = (W * [Xtest; ones(1,Ntest)]).';
nrmse = sqrt(mean((y_test - y_hat).^2)) / (max(y_test) - min(y_test));
fprintf('NARMA-10 NRMSE = %.4f\n', nrmse);
%% 8  Plot ------------------------------------------------------
figure;
plot(y_test,'k','LineWidth',1.2);  hold on;
plot(y_hat,'--','LineWidth',1.2);
legend('Target','Prediction');
title(sprintf('NARMA-10 – 100-node OEO, NRMSE = %.4f', nrmse));
xlabel('Sample'); ylabel('y'); grid on;
```

NARMA10 Vπ Sweep:

```
%% NARMA-10 prediction vs Vpi sweep
clear; close all;

%% 1  Data ------------------------------------------------------
Ntotal = 1000;
[~, y] = makeNARMA10(Ntotal, 42);
Ntrain = 800;   Ntest = Ntotal - Ntrain;
y_train = y(1:Ntrain);
y_test  = y(Ntrain+1:end);

%% 2  Reservoir parameters (fixed) ------------------------------
Qrf      = 100;
Glin_dB  = 0;
tauD     = 20e-6;      Fs = 2e9;
dt       = 1/Fs;
Ndelay   = round(tauD/dt);           % 40 000 samples
Nvirt    = 100;      group_len = Ndelay / Nvirt;

rng(1)
alpha = 1;
mask  = alpha * sign(randn(Nvirt,1));

%% 3  Pre-build the drive once ----------------------
drive = zeros(Ndelay*Ntotal,1);
for k = 1:Ntotal
    idx = (k-1)*Ndelay + (1:Ndelay);
    drive(idx) = encodeInput(y(k), mask, group_len);
end

%% 4  Sweep over Vpi values ---------------------------------------
Vpi_values = [0.2, 0.5, 2, 5, 7.5, 10];
nV = numel(Vpi_values);

nrmse_vals = zeros(1, nV);
W_cell     = cell(1, nV);

for iV = 1:nV
    Vpi = Vpi_values(iV);

    % 4.1  Run the OEO continuous simulation
    [~, y_res] = OEO(Qrf, Glin_dB, Vpi, drive);

    % 4.2  Slice out virtual-node taps
    node_idx = (0:Nvirt-1)*group_len + ceil(group_len/2);
    Ymat = reshape(y_res, Ndelay, Ntotal);
    Xall = Ymat(node_idx, :);
    Xtrain = Xall(:, 1:Ntrain);
    Xtest  = Xall(:, Ntrain+1:end);

    % 4.3  Ridge read-out
    ridge = 1e-6;
```

```
    W = (y_train.' * [Xtrain; ones(1,Ntrain)].') / ...
        ([Xtrain; ones(1,Ntrain)] * [Xtrain; ones(1,Ntrain)].' + ...
         ridge*eye(Nvirt+1));

    % 4.4  Predict & compute NRMSE
    y_hat = (W * [Xtest; ones(1,Ntest)]).';
    nrmse_vals(iV) = sqrt(mean((y_test - y_hat).^2)) ...
                     / (max(y_test) - min(y_test));

    W_cell{iV} = W;   % optional: keep the weights
    fprintf(' Vpi = %5.2f  →  NRMSE = %.4f\n', Vpi, nrmse_vals(iV));
end

%% 5  Plot summary of NRMSE vs Vpi ------------------------------
figure;
plot(Vpi_values, nrmse_vals, 'o-','LineWidth',1.5);
grid on;
xlabel('V_{\pi}');
ylabel('NRMSE');
title('NARMA-10 performance vs. V_{\pi}');
```

encodeInput.m

```
function u_long = encodeInput(u_scalar, mask, group_len)
% Duplicate a masked scalar into group_len samples per virtual node.
% u_scalar : one real number
% mask     : Nvirt × 1  (entries ±1 for a rich drive)
% group_len: samples per virtual node   (Ndelay = group_len·Nvirt)
   u_long = repelem(u_scalar .* mask, group_len);
end
```

makeNARMA10.m

```
function [u,y] = makeNARMA10(N,seed)
% NARMA-10 synthetic benchmark (length N)
% u   - drive,  0…0.5  uniform
% y   - target response
   if nargin<2, seed = 0; end
   rng(seed);
   u = 0.5*rand(N,1);
   y = zeros(N,1);
   for k = 10:N-1
       y(k+1) = 0.3*y(k)                          ...
              + 0.05*y(k)*sum(y(k-9:k))            ...
              + 1.5*u(k)*u(k-9) + 0.1;
   end
end
```

Sine vs Square Test: waveClassify_oeo_demo.m

```
%% Square vs Sine recognition with a 100-node OEO reservoir  –  continuous-run
clear; close all;
%% 1  Synthetic data ------------------------------------------------------
segLen   = 20;          % samples per snippet
NsegTot  = 50;          % number of snippets  (=> 1000 samples total)
[u, yLab] = makeSineSquare(NsegTot, segLen);      % helper below
Ntotal = numel(u);                    % 1000
pTrain = 0.8;        Ntrain = round(pTrain*Ntotal);
u_train = u(1:Ntrain);    y_train = yLab(1:Ntrain);
u_test  = u(Ntrain+1:end); y_test  = yLab(Ntrain+1:end);
%% 2  Reservoir parameters ------------------------------------------------
Qrf = 100;    Glin_dB = 0;    Vpi = 1;          % as before
tauD = 20e-6; Fs = 2e9;  dt = 1/Fs;
Ndelay = round(tauD/dt);                       % 40 000
Nvirt  = 100;   group_len = Ndelay/Nvirt;       % 400 samples / node
rng(1);  alpha = 1;                 % input scaling
mask = alpha * sign(randn(Nvirt,1));            % ±1 mask
%% 3  Build the long masked drive -----------------------------------------
drive = zeros(Ndelay*Ntotal,1);
for k = 1:Ntotal
  idx = (k-1)*Ndelay + (1:Ndelay);
  drive(idx) = encodeInput(u(k), mask, group_len);
end
%% 4  Single OEO pass -----------------------------------------------------
[~, yRes] = OEO(Qrf, Glin_dB, Vpi, drive);
%% 5  Tap the virtual nodes -----------------------------------------------
node_idx = (0:Nvirt-1)*group_len + ceil(group_len/2);
Ymat  = reshape(yRes, Ndelay, Ntotal);       % delay × time
Xall  = Ymat(node_idx, :);                   % Nvirt × time
Xtrain = Xall(:,1:Ntrain);     Xtest = Xall(:,Ntrain+1:end);
%% 6  Ridge read-out ------------------------------------------------------
ridge = 1e-6;
W = (y_train.' * [Xtrain; ones(1,Ntrain)].') / ...
  ([Xtrain; ones(1,Ntrain)]*[Xtrain; ones(1,Ntrain)].' + ...
   ridge*eye(Nvirt+1));
%% 7  Inference -----------------------------------------------------------
y_hat = (W * [Xtest; ones(1,numel(y_test))]).';   % raw scores
y_pred = y_hat > 0.5;                             % hard decision
acc = mean(y_pred == y_test);        % classification accuracy
fprintf('Wave-type accuracy = %.2f %%\n', 100*acc);
%% 8  Quick visual check --------------------------------------------------
figure;
plot(y_test,'k','LineWidth',1.2); hold on;                % target labels
stairs(y_hat,'r','LineWidth',1.1);                        % raw read-out score
plot(find(y_pred), y_pred(y_pred==1), 'bx', 'MarkerSize',7, 'LineWidth',1.4);
plot(find(~y_pred), y_pred(~y_pred), 'bx', 'MarkerSize',7, 'LineWidth',1.4);
legend({'Target (0 = sine, 1 = square)', ...
      'Read-out score', ...
      'Prediction'});
title(sprintf('Square / Sine – OEO reservoir  (accuracy %.2f%%)',100*acc));
xlabel('Sample index'); grid on;
```

makeSineSquare.m

```
function [u, lbl] = makeSineSquare(Nsegments, segLen)
% Fig. 3-style signal: random alternation of DC steps (square) and full-period sine
waves.
% lbl = 1 → step segment, lbl = 0 → sine segment.
ampStep = 1;                  % DC level for the step segments
ampSine = 1;                  % peak amplitude of the sine segments
u   = zeros(Nsegments*segLen,1);
lbl = zeros(size(u));
for s = 1:Nsegments
  idx = (s-1)*segLen + (1:segLen);
  if rand < 0.5
      % ---- sine segment ------------------------------------------------
      phi          = 0;                     % random phase
      u(idx)       = ampSine * sin(2*pi*(0:segLen-1)/segLen + phi);
      lbl(idx)     = 0;                              % 0 = sine
  else
      % ---- step (DC) segment -------------------------------------------
      u(idx)       = ampStep;                        % constant +1
      lbl(idx)     = 1;                              % 1 = step
  end
end
end
```

makeSineSquare_NoNoise.m:

```
function [u, lbl] = makeSineSquare_NoNoise(Nsegments, segLen)
% Random concatenation of one-period sines (label 0) and 0–1 square waves (label
1).
amp = 0.5;                              % common amplitude
u   = zeros(Nsegments*segLen ,1);
lbl = zeros(size(u));
tRel = (0:segLen-1).' / segLen;       % normalised time within a segment
for s = 1:Nsegments
   idx = (s-1)*segLen + (1:segLen);
   if rand < 0.5                      % --- sine segment ------------------
       u(idx)   = amp * sin(2*pi*tRel) + 0.5;
       lbl(idx) = 0;
   else                               % --- square-wave segment ----------
       u(idx)   = 2*amp * (tRel < 0.5);      % first half = 1, second half = 0
       lbl(idx) = 1;
   end
end
end
```

(No Noise) Sine vs Square test: classifySineSquare_NoNoise.m:

```
function classifySineSquare_NoNoise()
% Sine-vs-Square recognition with a 100-node OEO reservoir
% ---------------------------------------------------------
% Requires:  OEO.m, encodeInput.m (unchanged), makeSineSquare_NoNoise.m,
plotClassification.m
%% 1  synthetic dataset ------------------------------------------------
segLen   = 15;        % samples per snippet
NsegTot  = 50;        % number of snippets
pTrain   = 0.8;       % training fraction
[u, yLab] = makeSineSquare_NoNoise(NsegTot, segLen);
Ntotal = numel(u);
Ntrain = round(pTrain * Ntotal);
u_train = u(1:Ntrain);   y_train = yLab(1:Ntrain);
u_test  = u(Ntrain+1:end);  y_test = yLab(Ntrain+1:end);
%% 2  reservoir parameters ---------------------------------------------
Qrf = 100;     Glin_dB = 4;    Vpi  = 1;
tauD = 20e-6;  Fs      = 2e9;  dt   = 1/Fs;
Ndelay = round(tauD/dt);       % 40 000
Nvirt  = 100;                  % virtual nodes
group_len = Ndelay / Nvirt;    % 400 samples / node
rng(1);                        % reproducible mask
mask  = sign(randn(Nvirt,1));  % ±1
alpha = 1;  mask = alpha*mask;
%% 3  build the long masked drive --------------------------------------
drive = zeros(Ndelay * Ntotal,1);
for k = 1:Ntotal
   idx = (k-1)*Ndelay + (1:Ndelay);
   drive(idx) = encodeInput(u(k), mask, group_len);
end
%% 4  single OEO pass --------------------------------------------------
[~, yRes] = OEO(Qrf, Glin_dB, Vpi, drive);
%% 5  tap the virtual nodes --------------------------------------------
node_idx = (0:Nvirt-1)*group_len + ceil(group_len/2);
Ymat  = reshape(yRes, Ndelay, Ntotal);      % delay × time
Xall  = Ymat(node_idx, :);                  % Nvirt × time
Xtr   = Xall(:,1:Ntrain);   Xte = Xall(:,Ntrain+1:end);
%% 6  ridge read-out ---------------------------------------------------
ridge = 1e-6;
W = (y_train.' * [Xtr; ones(1,Ntrain)].') / ...
   ([Xtr; ones(1,Ntrain)]*[Xtr; ones(1,Ntrain)].' + ridge*eye(Nvirt+1));
%% 7  Inference  (unchanged) -------------------------------------------
y_hat  = (W * [Xte; ones(1,numel(y_test))]).';  % one score per *sample*
y_pred = y_hat > 0.5;                           % hard decision per sample
acc = mean(y_pred == y_test);
%% 8  Fig-3 style visualisation  (replace old block)
figure;
% top: input waveform
subplot(2,1,1);
plot(u_test,'b','LineWidth',1.1);  yline(0,'k:');
```

```
title('Input u(n): random sine and step segments');
xlabel('Sample index');  grid on;
% bottom: per-sample predictions
subplot(2,1,2);
stairs(y_test,'k','LineWidth',1.2); hold on;           % ground truth
plot(y_hat,'rx','MarkerSize',6,'LineWidth',1.1);       % raw scores
plot(y_pred,'bo','MarkerSize',4,'LineWidth',1.0);      % decisions
legend({'Target label (0 sine, 1 step)', ...
       'Reservoir score (per sample)', ...
       'Prediction (hard decision)'}, 'Location','best');
title(sprintf('Reservoir classification – sample accuracy %.2f %%',100*acc));
xlabel('Sample index');  grid on;
```

Plot Classification:

```
function plotClassification(u_test, y_test, y_hat_seg, y_pred_seg, segLen)
Nseg = numel(y_hat_seg);
tSeg = (0:Nseg-1)*segLen + ceil(segLen/2);
figure;
% --- top: input waveform ------------------------------------------------
subplot(2,1,1);
plot(u_test,'b','LineWidth',1.1);  yline(0,'k:');
title('Input u(n): random sine and step segments');
xlabel('Sample index');  grid on;
% --- bottom: labels vs reservoir output ---------------------------------
subplot(2,1,2);
stairs(y_test,'k','LineWidth',1.2); hold on;
plot(tSeg, y_hat_seg ,'rx','MarkerSize',8,'LineWidth',1.2);
plot(tSeg, y_pred_seg,'bo','MarkerSize',6,'LineWidth',1.1);
legend({'Target label (0 sine, 1 step)', ...
       'Reservoir score (segment mean)', ...
       'Prediction (hard decision)'}, 'Location','best');
title('Reservoir classification');
xlabel('Sample index');  grid on;
end
```

OEO Neural Network Parameter Sweep:

NARMA10 Sweep:

```
function narma10_oeo_demo()
% Parameter sweep for NARMA-10 prediction with the 100-node OEO reservoir
% ------------------------------------------------------------------------
% Sweeps
%   • Glin_dB = 0 : 9      (10 values)
%   • Vpi     = 0.3 : 0.3 : 4.5   (15 values)
%     – Switch to linspace(0.3,5,15) if the top point must be 5 V exactly.
%
% The script calls NARMA_once() which reproduces the original
% "continuous-run" experiment while taking Glin_dB and Vpi as inputs.
% The output NRMSE matrix is 10×15.

% 1 – Sweep vectors ------------------------------------------------------
Gvec = 0:9;                % 10 values for Glin_dB
Vvec = 0.3:0.3:4.5;        % 15 values for Vpi

NRMSE = zeros(numel(Gvec), numel(Vvec));

% 2 – Nested sweep -------------------------------------------------------
for ig = 1:numel(Gvec)
    for iv = 1:numel(Vvec)
        NRMSE(ig,iv) = NARMA_once(Gvec(ig), Vvec(iv));
    end
end

% 3 – Heat-map -----------------------------------------------------------
figure;
imagesc(Vvec, Gvec, NRMSE);
axis xy;
colorbar;
colormap(jet);
xlabel('V_{\pi} (V)');
ylabel('G_{lin} (dB)');
title('NARMA-10 NRMSE vs. G_{lin} and V_{\pi}');
end

% ========================================================================
function nrmse = NARMA_once(Glin_dB, Vpi)
% One complete NARMA-10 prediction run with given OEO parameters
Ntotal = 1000;                      % samples to predict
[~, y] = makeNARMA10(Ntotal, 42);    % reproducible series

Ntrain = 800;   Ntest = Ntotal - Ntrain;
y_train = y(1:Ntrain);
```

```
y_test  = y(Ntrain+1:end);

% Reservoir parameters ------------------------------------------------
Qrf      = 100;
% Glin_dB  = input argument
% Vpi      = input argument
tauD     = 20e-6;      Fs = 2e9;
dt       = 1/Fs;
Ndelay   = round(tauD/dt);           % 40 000 samples
Nvirt    = 100;      group_len = Ndelay / Nvirt;   % 400 samples/node

rng(1)
alpha = 1;
mask  = alpha * sign(randn(Nvirt,1));

% Build masked drive --------------------------------------------------
drive = zeros(Ndelay*Ntotal,1);
for k = 1:Ntotal
    idx = (k-1)*Ndelay + (1:Ndelay);
    drive(idx) = encodeInput(y(k), mask, group_len);
end

% Run OEO -------------------------------------------------------------
[~, y_res] = OEO(Qrf, Glin_dB, Vpi, drive);

% Tap virtual nodes ---------------------------------------------------
node_idx = (0:Nvirt-1)*group_len + ceil(group_len/2);
Ymat = reshape(y_res, Ndelay, Ntotal);
Xall = Ymat(node_idx, :);
Xtrain = Xall(:, 1:Ntrain);
Xtest  = Xall(:, Ntrain+1:end);

% Ridge read-out ------------------------------------------------------
ridge = 1e-6;
W = (y_train.' * [Xtrain; ones(1,Ntrain)].') / ...
    ([Xtrain; ones(1,Ntrain)] * [Xtrain; ones(1,Ntrain)].' + ...
     ridge*eye(Nvirt+1));

y_hat = (W * [Xtest; ones(1,Ntest)]).';

nrmse = sqrt(mean((y_test - y_hat).^2)) / (max(y_test) - min(y_test));
end
```

SineSquare Sweep:

```
function classifySineSquare_NoNoise()
% Parameter sweep for sine-vs-square classification using the 100-node OEO reservoir
% -------------------------------------------------------------------------
% Sweeps
%   • Glin_dB = 1 : 10   (10 values)
%   • Vpi     = 0.3 : 0.3 : 4.5   (15 values, matches the 0.3-increment list)
%     – If you really need to include Vpi = 5 as the last point, replace the
%       Vpi vector by linspace(0.3,5,15).
%
% The script calls a helper function CLASSIFY_ONCE that keeps the original
% classifySineSquare_NoNoise() logic unchanged except that Glin_dB and Vpi
% are supplied as input arguments.  The output ACC is a 10×15 matrix whose
% (i,j) entry is the sample-level accuracy for Glin_dB(i) & Vpi(j).
%   1── sweep vectors ------------------------------------------------------
Gvec = 1:10;                      % 10 values
Vvec = 0.3:0.3:4.5;              % 15 values (0.3,0.6,…,4.5)
ACC  = zeros(numel(Gvec), numel(Vvec));
%   2── nested sweep -------------------------------------------------------
for ig = 1:numel(Gvec)
   for iv = 1:numel(Vvec)
       ACC(ig,iv) = classify_once(Gvec(ig), Vvec(iv));
   end
end
%   3── quick visual check -------------------------------------------------
figure;
imagesc(Vvec, Gvec, ACC*100);
axis xy;
colorbar;
colormap(jet);
xlabel('V_{\pi} (V)');
ylabel('G_{lin} (dB)');
title('Sample-level accuracy [%] vs. G_{lin} and V_{\pi}');
end
% ========================================================================
function acc = classify_once(Glin_dB, Vpi)
% ----- core routine (verbatim copy of the original with 2 parameters) ----
segLen   = 15;        % samples per snippet
NsegTot  = 50;        % number of snippets
pTrain   = 0.8;       % training fraction
[u, yLab] = makeSineSquare_NoNoise(NsegTot, segLen);
Ntotal = numel(u);
Ntrain = round(pTrain * Ntotal);
u_train = u(1:Ntrain);   y_train = yLab(1:Ntrain);
u_test  = u(Ntrain+1:end);  y_test = yLab(Ntrain+1:end);
Qrf   = 100;     % keeps original value
% ── the two swept parameters come in here ──
% Glin_dB = input argument
% Vpi     = input argument
tauD = 20e-6;  Fs = 2e9;  dt = 1/Fs;
```

```
Ndelay = round(tauD/dt);       % 40 000
Nvirt  = 100;                  % virtual nodes
group_len = Ndelay / Nvirt;    % 400 samples / node
rng(1);                        % reproducible mask
mask  = sign(randn(Nvirt,1));
alpha = 1;  mask = alpha*mask;
% build long masked drive
drive = zeros(Ndelay * Ntotal,1);
for k = 1:Ntotal
   idx = (k-1)*Ndelay + (1:Ndelay);
   drive(idx) = encodeInput(u(k), mask, group_len);
end
% single OEO pass
[~, yRes] = OEO(Qrf, Glin_dB, Vpi, drive);
% tap virtual nodes
node_idx = (0:Nvirt-1)*group_len + ceil(group_len/2);
Ymat  = reshape(yRes, Ndelay, Ntotal);
Xall  = Ymat(node_idx, :);
Xtr   = Xall(:,1:Ntrain);   Xte = Xall(:,Ntrain+1:end);
% ridge read-out
ridge = 1e-6;
W = (y_train.' * [Xtr; ones(1,Ntrain)].') / ...
   ([Xtr; ones(1,Ntrain)]*[Xtr; ones(1,Ntrain)].' + ridge*eye(Nvirt+1));
% inference
y_hat  = (W * [Xte; ones(1,numel(y_test))]).';
y_pred = y_hat > 0.5;
acc = mean(y_pred == y_test);
end
```